/03

语言模型

语言模型

N元语法

中文分词语料库

一元语法分词

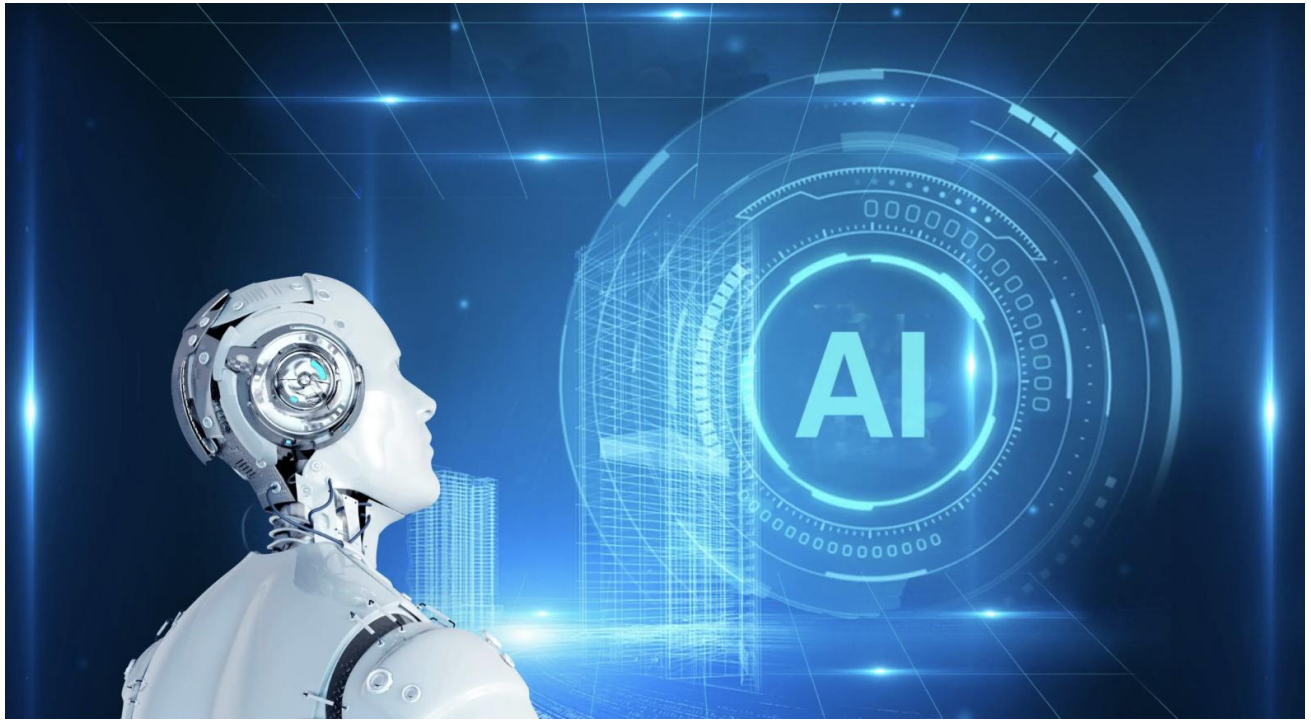
语言模型

语言模型目标是建模自然语言的概率分布，在自然语言处理研究中具有重要的作用，是机器翻译、语音识别、输入法、句法分析等任务的支撑。语言模型是自然语言处理基础任务之一，大量的研究从n元语言模型和神经语言模型等不同角度开展了系列工作。

由于语言模型可以为自然语言的表示学习提供天然的自监督训练目标，近年来，**预训练语言模型**做为通用的基于深度神经网络的自然语言处理算法的基础工具，受到越来越多的重视。大规模的预训练语言模型对于提升各类自然语言处理任务的效果起到了重要作用。

语言模型

语言模型从人类产生的数据中学习，这些数据可能包含性别歧视、地域偏见、网络谣言等。技术上的“稀疏”尚可通过平滑弥补，但社会性的“偏见”若被模型习得并放大，将固化社会歧视，产生真实伤害。这要求我们超越纯技术视角，建立技术伦理的**敏感性**。



统计语言模型

- 从统计角度看，自然语言中的一个句子s可以由任何词串构成。不过概率$P(s)$有大有小。如：

$$s1 = \text{我刚吃过晚饭}$$

$$s2 = \text{刚我过晚饭吃}$$

$$p(s1) > p(s2)$$

- 对于给定的自然语言而言，通常$p(s)$是未知的。
- 对于一个服从某个未知概率分布p的语言L，根据给定的语言样本估计p的过程被称作语言建模。
- 根据语言样本估计出的概率分布p就称为语言L的语言模型。

统计语言模型

- 语言建模技术首先在语音识别研究中提出，后来陆续用到OCR、手写体识别、机器翻译、信息检索等领域。
- 在语音识别中，如果识别结果有多个，则可以根据语言模型计算每个识别结果的可能性，然后挑选一个可能性较大的识别结果。
- 汉语切分歧义消解?

香农游戏(Shannon Game)

- Claude E. Shannon. “Prediction and Entropy of Printed English”, Bell System Technical Journal 30:50-64. 1951.
- 给定前n-1个词(或者字母)，预测下一个词(字母)
- 从训练语料库中确定不同词序列概率

统计语言模型

- **微型语料库**

商品/和/服务
商品/和服/物美价廉
服务/和/货币

商品	2	n	
和	2	n	
服务	2	n	
和服	1	n	
物美价廉		1	n
货币	1	n	

一个朴素的语言模型：

$p(\text{商品 和 服务}) = \frac{1}{3}$、$p(\text{商品 和服 物美价廉}) = \frac{1}{3}$、$p(\text{服务 和 货币}) = \frac{1}{3}$、其他句子的概率都为0

语言模型

- 把句子表示为单词列表$\boldsymbol{w} = w_1 w_2 ... w_k$，每个$w_t, t \in [1, k]$都是一个单词，然后定义语言模型：

$$
\begin{aligned}
p(\mathbf{w}) &= p(w_1 w_2 \ldots w_k) \\
&= p(w_1|w_0) \times p(w_2|w_0 w_1) \times \ldots \times p(w_{k+1}|w_0 w_1 w_2 \ldots w_k) \\
&= \prod_{t=1}^{k+1} p(w_t|w_0 w_1 \ldots w_{t-1})
\end{aligned}
\tag{1}
$$

怎么推导的?

其中w_0和w_{k+1}为句子开头和结尾。

$$p(A|B) = \frac{p(AB)}{p(B)} \quad \Rightarrow \; p(w_1 w_2 \cdots w_k) = p(w_k|w_1 w_2 \cdots w_{k-1}) p(w_1 w_2 \cdots w_{k-1})$$

令$A = w_k$, $B = w_1 w_2 \cdots w_{k-1}$

建立n-gram

- **n元语法**（n-gram）的定义：每个单词的概率仅取决于该单词之前的$n-1$个单词。也即：

$$p(\boldsymbol{w}) \quad = \prod_{t=1}^{k+n-1} p(w_t | w_{t-n+1} \ldots w_{t-1})$$

- 当$n=1$时的n元语法称为一元语法（unigram）
- 当$n=2$时的n元语法称为二元语法（bigram）
- 当$n=3$时的n元语法称为三元语法（trigram）
- $n>=4$时数据稀疏和计算代价又变得显著起来，实际工程中几乎不使用。
- 另外，深度学习中的递归神经网络语言模型（RNN Language Model），理论上可以记忆无限个单词，可以看作“无穷元语法”（∞ –gram）。

建立n-gram

- 数据准备:
  - 确定训练语料
  - 对语料进行分词
  - 句子边界，增加两个特殊的词<BOS>和<EOS>
    - I eat . <BOS> I eat . <EOS>
    - I sleep . <BOS> I sleep . <EOS>
- 参数估计
  - 利用训练语料，估计模型参数

建立n-gram

- 最大似然估计（Maximum Likelihood Estimates，MLE）

  选择参数使得训练语料出现的概率最大，用实际样本中事件的频率估计该事件的概率

$$p(w_t|w_0 \ldots w_{t-1}) = p_{\mathrm{ML}}(w_t|w_0 \ldots w_{t-1}) = \frac{c(w_0 \ldots w_{t-1} w_t)}{c(w_0 \ldots w_{t-1})}$$

其中，$c(w_0 \ldots w_t)$表示$w_0 \ldots w_t$的计数（count），比如

$$p(\text{商品 和 服务}) = p(\text{商品}|\mathrm{BOS})p(\text{和}|\mathrm{BOS}\,\text{商品})p(\text{服务}|\mathrm{BOS}\,\text{商品 和})p(\mathrm{EOS}|\mathrm{BOS}\,\text{商品 和 服务})$$

$$= \frac{2}{3} \times \frac{1}{2} \times \frac{1}{1} \times \frac{1}{1} = \frac{1}{3}$$

案例展示--模型的“天真”假设

原句：小明昨天买的那辆在车展上获奖的、由德国工程师设计、配备了最新电动系统的自行车，今天早上被他骑到公园去了。

我们使用一个trigram模型 (n=3) 来预测句子的下一个词（基于常见语料训练的概率）。我们要预测谓语部分（“...自行车，今天早上被他骑...”中的“骑”）

结果发现“被他”后“发现”、“抓住”、“叫”等（概率高），而“骑”的概率较低。

案例展示--模型的“天真”假设

Q1:为什么模型会做出和我们直觉相反的预测?

Q2:我们该如何看待模型与现实世界的关系?

N的选择

- 词表中词的个数 |V| = 20,000 词

n	所有可能的n-gram的个数
2 (bigrams)	400,000,000
3 (trigrams)	8,000,000,000,000
4 (4-grams)	1.6×10^{17}

可靠性 vs. 辨别力

- n 较大时
  - 提供了更多的语境信息，语境更具区别性，但是，参数个数多、计算代价大、训练语料需要多、参数估计不可靠。
- n 较小时
  - 语境信息少，不具区别性。 但是，参数个数少、计算代价小、训练语料无需太多、参数估计可靠。

一元语法

- 利用一元语法模型，计算“商品 和 服务”的概率

$$p(\text{商品 和 服务}) = p(\text{商品})p(\text{和})p(\text{服务})$$
$$= \frac{2}{9} \times \frac{2}{9} \times \frac{2}{9}$$

$$p(\text{商品 和服 务}) = p(\text{商品})p(\text{和服})p(\text{务})$$
$$= \frac{2}{9} \times \frac{1}{9} \times \frac{0}{9}$$

商品	2	n
和	2	n
服务	2	n
和服	1	n
物美价廉	1	n
货币	1	n

二元语法

- 每次计算只涉及连续两个单词的二元接续，

$$
\begin{aligned}
p(\mathbf{w}) &= p(w_1 w_2 \ldots w_k) \\
&= p(w_1|w_0) \times p(w_2|w_1) \times \ldots \times p(w_{k+1}|w_k) \\
&= \prod_{t=1}^{k+1} p(w_t|w_{t-1})
\end{aligned}
\tag{1}
$$

二元语法

- 利用二元语法模型，计算“商品 和 服务”的概率

$$
\begin{aligned}
p(\text{商品和服务}) &= p(\text{商品}|\text{BOS})p(\text{和}|\text{商品})p(\text{服务}|\text{和})p(\text{EOS}|\text{服务}) \\
&= \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{1} \\
&= \frac{1}{6}
\end{aligned}
$$

- 新句子p(商品 和 货币)

$$
\begin{aligned}
p(\text{商品和货币}) &= p(\text{商品}|\text{BOS})p(\text{和}|\text{商品})p(\text{货币}|\text{和})p(\text{EOS}|\text{货币}) \\
&= \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{1} \\
&= \frac{1}{6}
\end{aligned}
$$

平滑

尽管 n 元语言模型能缓解句子概率为 0 的问题，但语言是由人和时代创造的，具备无穷的可能性，再庞大的训练语料也无法覆盖所有的 n-gram，而训练语料中的零频率并不代表零概率。因此，需要使用平滑技术（Smoothing）来解决这一问题。

平滑是指为了产生更合理的概率，对最大似然估计进行调整的一类方法，也称为数据平滑（Data Smoothing）。**平滑处理的基本思想**是提高低概率，降低高概率，使整体的概率分布趋于均匀。

平滑

在算法设计中应有意识地**为“少数群体”或“不同声音”保留空间**，主动规避主流数据带来的偏见。技术人员不仅是模型的构建者，更应是数字公平的“守门人”。通过主动的数据清洗、偏见检测和算法干预，将**社会责任**内嵌于技术方案之中。

加法平滑

加法平滑（Additive Smoothing）是实际运用中最常用的平滑技术之一。其思想是假设事件出现的次数比实际出现的次数多δ次。

以二元语法模型为例，其平滑后的条件概率为：

$$p(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + \delta}{C(w_{i-1}) + |V|\delta}$$

其中0 ⩽ δ ⩽ 1，V是训练语料中所有单词的集合，$C(w_i)$ 表示单词w_i 出现的次数，$C(w_{i-1}w_i)$代表词w_{i-1}和词w_i同时出现次数。可以进一步拓展到n元语言模型上：

$$p(w_i|w_{i-1}) = \frac{C\left(w_{i-n+1}^{i}\right) + \delta}{C\left(w_{i-n+1}^{i-1}\right) + |V|\delta}$$

当δ = 1 时，该方法又称为加一平滑。

加一法

对于二元语法来说，最简单的平滑技术就是假设每个二元语法出现的次数比实际多一次，于是

$$p(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + |V|}$$

其中V是考虑所有词汇的单词表。

古德-图灵估计法

古德-图灵估计法是1953年由I. J. Good基于图灵(Turning)的方法提出的，是多种平滑技术的核心。该方法基于的核心思想是将一部分已知事件的概率分配给未见的事件。

对于n元语言模型来说，降低出现次数多的n-gram，同时将剩余概率分配给未出现的n-gram。具体来说，对于任意一个出现了r次的n元语法，按照如下公式修改为出现了r^*次

$$r^* = (r+1)\frac{N_{r+1}}{N_r}$$

其中，N_r 代表在训练语料中出现 r 次的n-gram 数量。对其进行归一化后，即可得到出现 r 次的 n 元语法概率

$$p_r = \frac{r^*}{N}$$

其中$N = \sum_{r>0} N_r * r$

古德-图灵估计法

那么对于 $r = 0$（未观察到的）的样本

$$
\begin{aligned}
p_0 &= 1 - \sum_{r>0} N_r * p_r \\
&= 1 - \frac{1}{N} \sum_{r>0} N_r * r^* \\
&= \frac{N - \sum_{r>0} N_r * r^*}{N} \\
&= \frac{N - \sum_{r>0} N_{r+1} * (r+1)}{N} \\
&= \frac{N - (N - N_1)}{N} \\
&= \frac{N_1}{N}
\end{aligned}
$$

因此，对于$r = 0$ 的未见事件，有N_1/N 的概率余量可以用于分配。

古德-图灵估计法

r	n_r	r^*	p_r
1	3286	0.448	5.220×10^{-5}
2	736	1.25	1.454×10^{-4}
3	306	2.25	2.620×10^{-4}
4	172	3.17	3.693×10^{-4}
5	109	4.18	4.875×10^{-4}
6	76	5.53	6.440×10^{-4}
7	60	5.73	6.681×10^{-4}
8	43	5.86	6.830×10^{-4}
9	28	7.14	8.324×10^{-4}
10	20	–	–

Katz平滑

Katz平滑是1987年由S. M. Katz所提出的后备（back-off）平滑方法，其在古德-图灵估计法的基础上引入了高阶模型与低阶模型的结合。**Katz平滑法的基本思想**是将因减值获得的概率余量根据低阶模型的分布分配给未见事件，而不是进行平均分配，从而令低概率事件有更合理的概率分布。

Katz平滑法的做法是，当事件在样本中出现的**频次大于某一数值k时**，运用最大似然估计法，通过减值来估计其概率值；而当事件的**频次小于k值时**，使用低阶的语法模型作为代替高阶语法模型的后备。

对于一个高阶n元组 $w_1, w_2, \ldots, w_n$ ，其概率 $P(w_n | w_1, w_2, \ldots, w_{n-1})$的计算公式为：

$$P_{\text{Katz}}(w_n \mid w_1, w_2, \ldots, w_{n-1}) = \begin{cases} d_r \cdot \frac{C(w_1, w_2, \ldots, w_n)}{C(w_1, w_2, \ldots, w_{n-1})} & \text{如果 } C(w_1, w_2, \ldots, w_n) > 0, \\ \alpha(w_1, w_2, \ldots, w_{n-1}) \cdot P_{\text{Katz}}(w_n \mid w_2, w_3, \ldots, w_{n-1}) & \text{如果 } C(w_1, w_2, \ldots, w_n) = 0. \end{cases}$$

Katz平滑

折扣因子 d_r 的计算

Katz平滑使用Good - Turing估计来计算折扣因子。对于一个出现次数为r的n元组，折扣因子的计算公式为：

$$d_r = \frac{(r+1)N_{r+1}}{rN_r}$$

回退权重 $\alpha(w_1, w_2, \ldots, w_{n-1})$ 的计算

回退权重 $\alpha(w_1, w_2, \ldots, w_{n-1})$ 用于将剩余的概率空间分配给低阶模型。其计算公式为：

$$\alpha(w_1, w_2, \ldots, w_{n-1}) = \frac{1 - \sum_{\text{all observed } w_n} d_r \cdot \frac{C(w_1, w_2, \ldots, w_n)}{C(w_1, w_2, \ldots, w_{n-1})}}{1 - \sum_{\text{all observed } w_n} P_{\text{Katz}}(w_n \mid w_2, w_3, \ldots, w_{n-1})}$$

回退权重的作用是确保概率分布的完整性，即所有概率的总和为1。

平滑方法总结

除了我们在上述平滑算法之外，研究人员们提出了有很多针对语言模型的平滑算法，包括Jelinek Mercer平滑算法、Witten-Bell平滑算法、Kneser-Ney平滑算法等。这些方法的核心思想大都可以归纳为：**如果n-gram存在则使用其本身计数，如果不存在再退后到低阶分布。**可以用如下公式表示

$$P_{\text{smooth}}(w_i|w_{i-n+1}^{i-1}) = \begin{cases} \alpha(w_i|w_{i-n+1}^{i-1}), & \text{如果 } C(w_{i-n+1}^{i}) > 0 \\ \gamma(w_{i-n+1}^{i-1})P_{\text{smooth}}(w_i|w_{i-n+2}^{i-1}), & \text{如果 } C(w_{i-n+1}^{i}) = 0 \end{cases}$$

意思是，如果某个n-gram的计数大于0，就使用它的调整概率α；如果计数为0，就使用一个折扣因子γ乘以低一阶的平滑概率。

平滑策略

在此基础上，一些平滑算法采用高阶和低阶n元语法模型的线性插值的方法，融合高阶和低阶语法的估计

$$P_{\text{smooth}}(w_i|w_{i-n+1}^{i-1}) = \lambda P_{\text{ML}}(w_i|w_{i-n+1}^{i-1}) + (1-\lambda)P_{\text{smooth}}(w_i|w_{i-n+2}^{i-1})$$

- 新的二元语法概率为：

$$p(w_t|w_{t-1}) = \lambda p_{\text{ML}}(w_t|w_{t-1}) + (1-\lambda)p(w_t)$$

$\lambda \in (0,1)$为常数平滑因子。

语言模型

语言模型在中文分词中需精准处理多义、歧义现象，这可关联信息传播的社会责任。技术应用于中文处理时，需秉持严谨态度，保障信息准确传达，尤其在涉及国家政策、社会热点时，要坚守信息真实性原则，培养专业伦理与**社会责任感**，避免技术应用引发信息偏差问题。这要求未来的工程师必须坚守“产品即责任”的职业操守。在模型设计、开发和部署的每一个环节，都要预判并评估其可能的社会影响，确保自己的技术成果被用于**弘扬正气、促进和谐**，而非传播谬误、放大偏见。

1998年《人民日报》语料库PKU

- 1998年全年2600万汉字的「人民日报标注语料库」
- 有偿公开现已完成的1998年上半年的「人民日报标注语料库」 (约1,300万字＝约730万词)
- 免费公开1个月的「人民日报标注语料库」,欢迎广大研究人员自由下载

微软亚洲研究院语料库MSR

1) 标注一致性上MSR要优于PKU，这一点可以通过历史文献报告的准确率佐证。

2) 切分颗粒度上MSR要大于PKU，MSR的机构名称不予切分，而PKU则拆开了。

3) MSR中姓名作为一个整体，更符合习惯。

4) MSR量级是PKU的两倍。

繁体中文分词语料库

- 香港城市大学提供的CITYU
- 台湾中央研究院提供的AS

语料库统计

表3-1 中文分词语料库训练集统计

语料库	字符数	词语种数	总词频	平均词长
PKU	183万	6万	111万	1.6
MSR	405万	9万	237万	1.7
AS	837万	14万	545万	1.5
CITYU	240万	7万	146万	1.7

表3-2 中文分词语料库测试集统计

语料库	字符数	词语种数	总词频	平均词长	OOV
PKU	17万	1万	10万	1.7	5.75%
MSR	18万	1万	11万	1.7	2.65%
AS	20万	2万	12万	1.6	4.33%
CITYU	7万	1万	4万	1.7	7.40%

Jieba分词

jieba分词主要是基于统计词典，构造一个前缀词典；然后利用前缀词典对输入句子进行切分，得到所有的切分可能，根据切分位置，构造一个有向无环图；通过动态规划算法，计算得到最大概率路径，也就得到了最终的切分形式。

前缀词典构建

以“去北京大学玩”为例，作为待分词的输入文本。

离线统计的词典形式如下，每一行有三列，第一列是词，第二列是词频，第三列是词性。

北京大学 2053 nt
大学 20025 n
去 123402 v
玩 4207 v
北京 34488 ns
北 17860 ns
京 6583 ns
大 144099 a
学 17482 n

前缀词典构建

首先是基于统计词典构造前缀词典，如统计词典中的词“北京大学”的前缀分别是“北”、“北京”、“北京大”；词“大学”的前缀是“大”。统计词典中所有的词形成的前缀词典如下所示，你也许会注意到“北京大”作为“北京大学”的前缀，但是它的词频却为0，这是为了便于后面有向无环图的构建。

北京大学 2053
北京大 0
大学 20025
去 123402
玩 4207
北京 34488
北 17860
京 6583
大 144099
学 17482

前缀词典构建

```
def gen_pfdict(f):
    lfreq = {}
    ltotal = 0
    for lineno, line in enumerate(f, 1):
        try:
            line = line.strip()
            word, freq = line.split(' ')[:2]
            freq = int(freq)
            lfreq[word] = freq
            ltotal += freq
            for ch in range(len(word)):
                wfrag = word[:ch + 1]
                if wfrag not in lfreq:
                    lfreq[wfrag] = 0 # 不在词典里的前缀，词频设为零，前提是词典已经进行排序
        except ValueError:
            raise ValueError(
                'invalid dictionary entry in %s at Line %s: %s' % (f.name, lineno, line))
    f.close()
    return lfreq, ltotal
```

注意没有先读取整个词表，再判断前缀是否在词典中

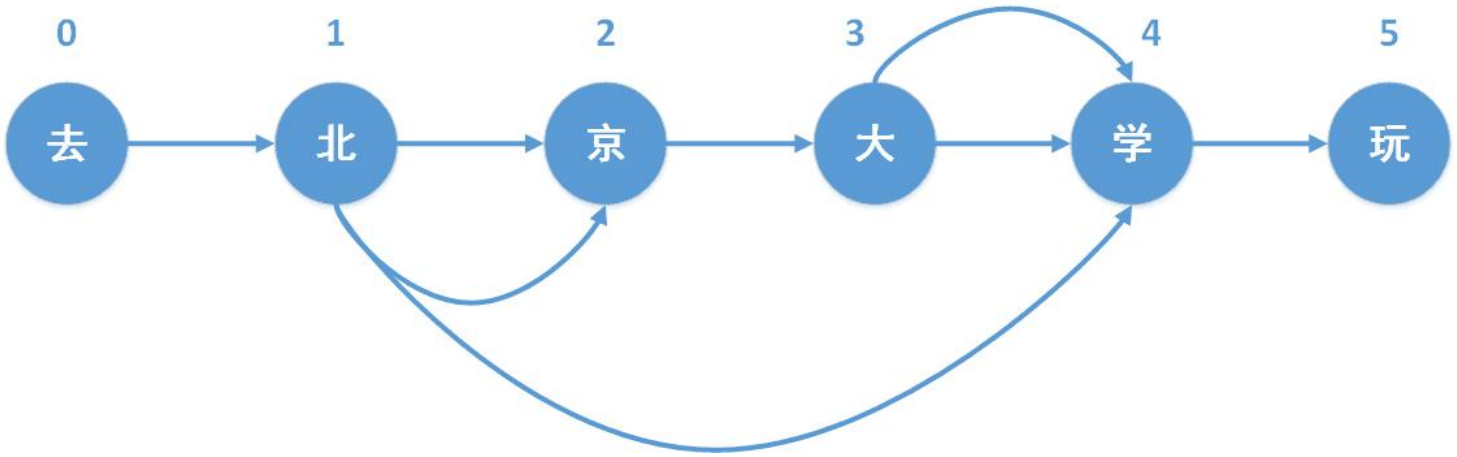
有向无环图构建

有向无环图，directed acyclic graphs，简称DAG，是一种图的数据结构，顾名思义，就是没有环的有向图。

基于前缀词典，对输入文本进行切分，对于“去”，没有前缀，那么就只有一种划分方式；对于“北”，则有“北”、“北京”、“北京大学”三种划分方式；对于“京”，也只有一种划分方式；对于“大”，则有“大”、“大学”两种划分方式，依次类推，可以得到每个字开始的前缀词的划分方式。

在jieba分词中，对每个字都是通过在文本中的位置来标记的，因此可以构建一个以位置为key，相应划分的末尾位置构成的列表为value的映射

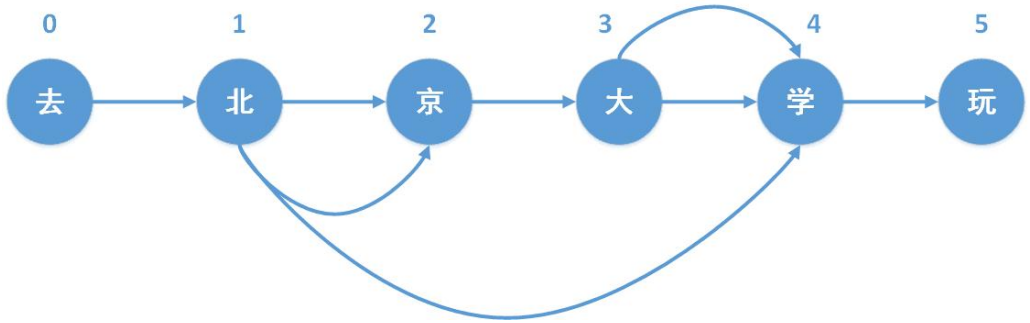
0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]



有向无环图构建

```python
def get_DAG(sentence, FREQ):
    DAG = {}
    N = len(sentence)
    for k in range(N):
        tmplist = []
        i = k
        frag = sentence[k]
        while i < N and frag in FREQ:
            if FREQ[frag]:
                tmplist.append(i)
            i += 1
            frag = sentence[k:i + 1]
        if not tmplist:
            tmplist.append(k)
        DAG[k] = tmplist
    return DAG
```

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]



最大概率路径计算

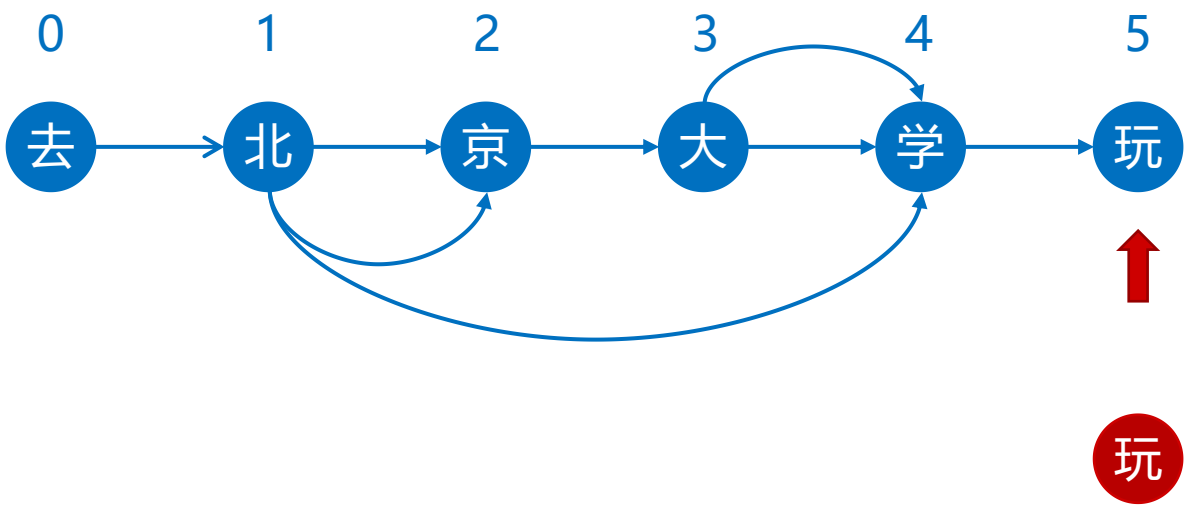
在得到所有可能的切分方式构成的有向无环图后，我们发现从起点到终点存在多条路径，多条路径也就意味着存在多种分词结果

序号	路径	分词结果
1	0 -> 1 -> 2 -> 3 -> 4 -> 5	去 / 北 / 京 / 大 / 学 / 玩
2	0 -> 1 , 2 -> 3 -> 4 -> 5	去 / 北京 / 大 / 学 / 玩
3	0 -> 1 , 2 -> 3 , 4 -> 5	去 / 北京 / 大学 / 玩
4	0 -> 1 , 2 , 3 , 4 -> 5	去 / 北京大学 / 玩

最大概率路径计算

最大概率路径计算是一个自底向上的动态规划问题，它从sentence的最后一个字（N-1）开始倒序遍历sentence的每个字（idx），计算子句sentence[idx ~ N-1]的概率对数得分。然后将概率对数得分最高的情况以元组（概率对数，词语最后一个位置）保存在route中。

最大概率路径计算



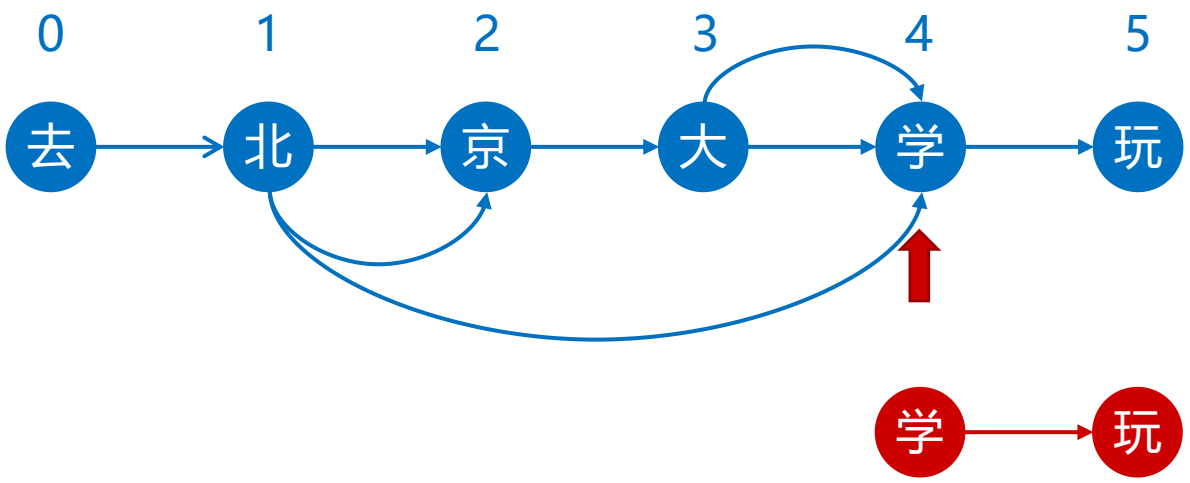
Rounte = {6: (0, 0),
5: (-9.57, 5)
}

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

$p(\text{玩})$ = log(4207)-log(60101967)

=-9.57

最大概率路径计算



Rounte = {6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4)
}

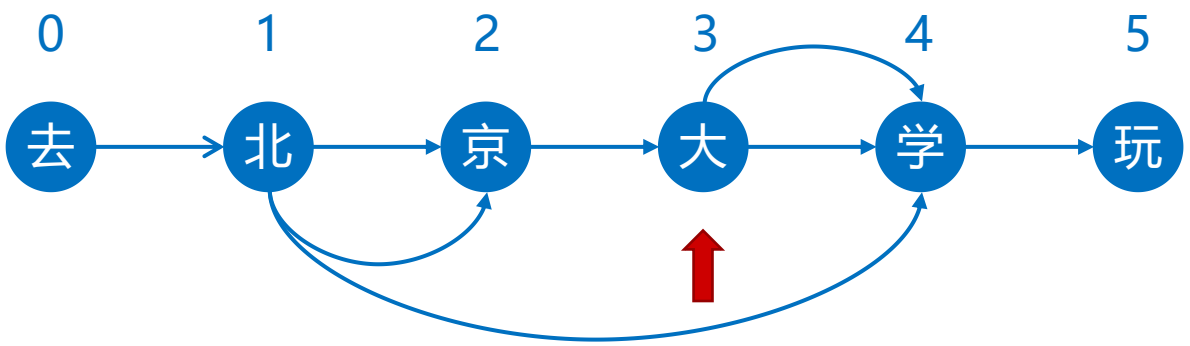
p(学) =-8.14

p(玩) =-9.57

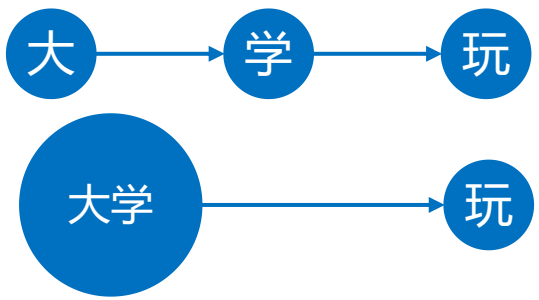
p(学 → 玩) =−17.71

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

最大概率路径计算



Rounte = {6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4)
}



$p(\text{大}) = -6.03$

$p(\text{大} \rightarrow \text{学} \rightarrow \text{玩}) =- 23.74$

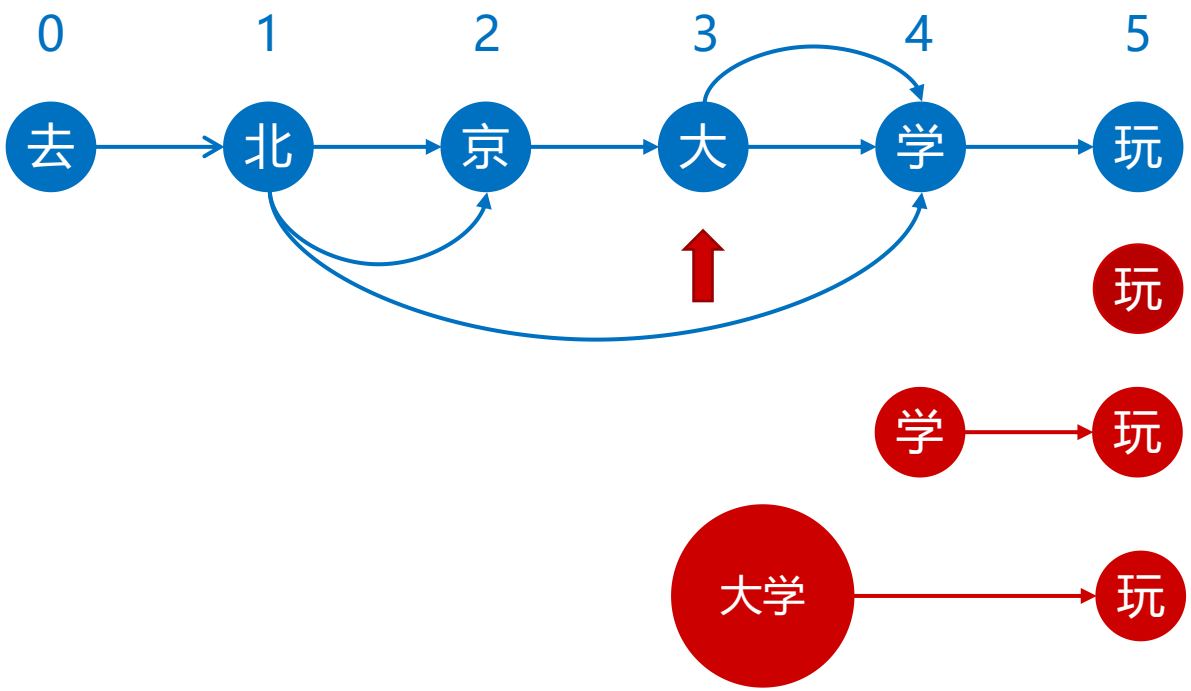
$p(\text{大学}) =-8.01$

$p(\text{大学} \rightarrow \text{玩}) =- 17.58$

$p(\text{大学} \rightarrow \text{玩}) > p(\text{大} \rightarrow \text{学} \rightarrow \text{玩})$

0: [0]
1: [1,2,4]
2: [2]
➡ 3: [3,4]
4: [4]
5: [5]

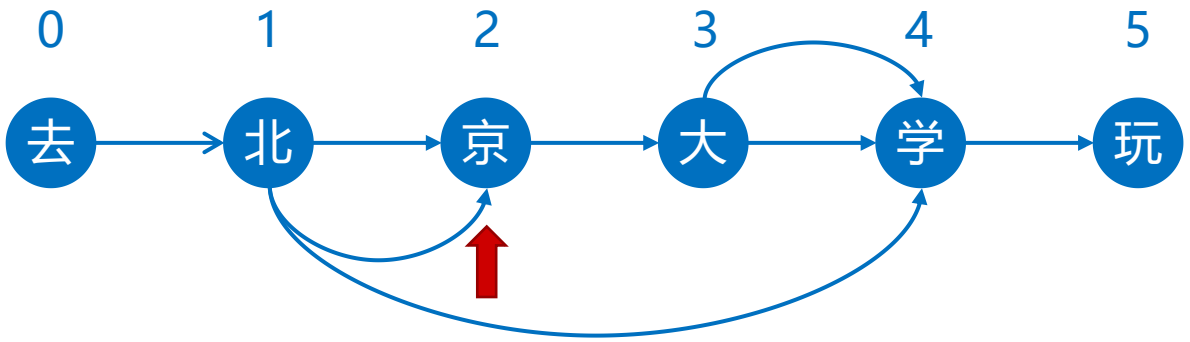
最大概率路径计算



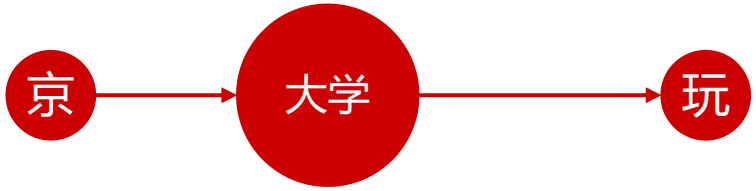
Rounte = {6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4)
}

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

最大概率路径计算



Rounte ={6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4),
2: (-26.7, 2),
}

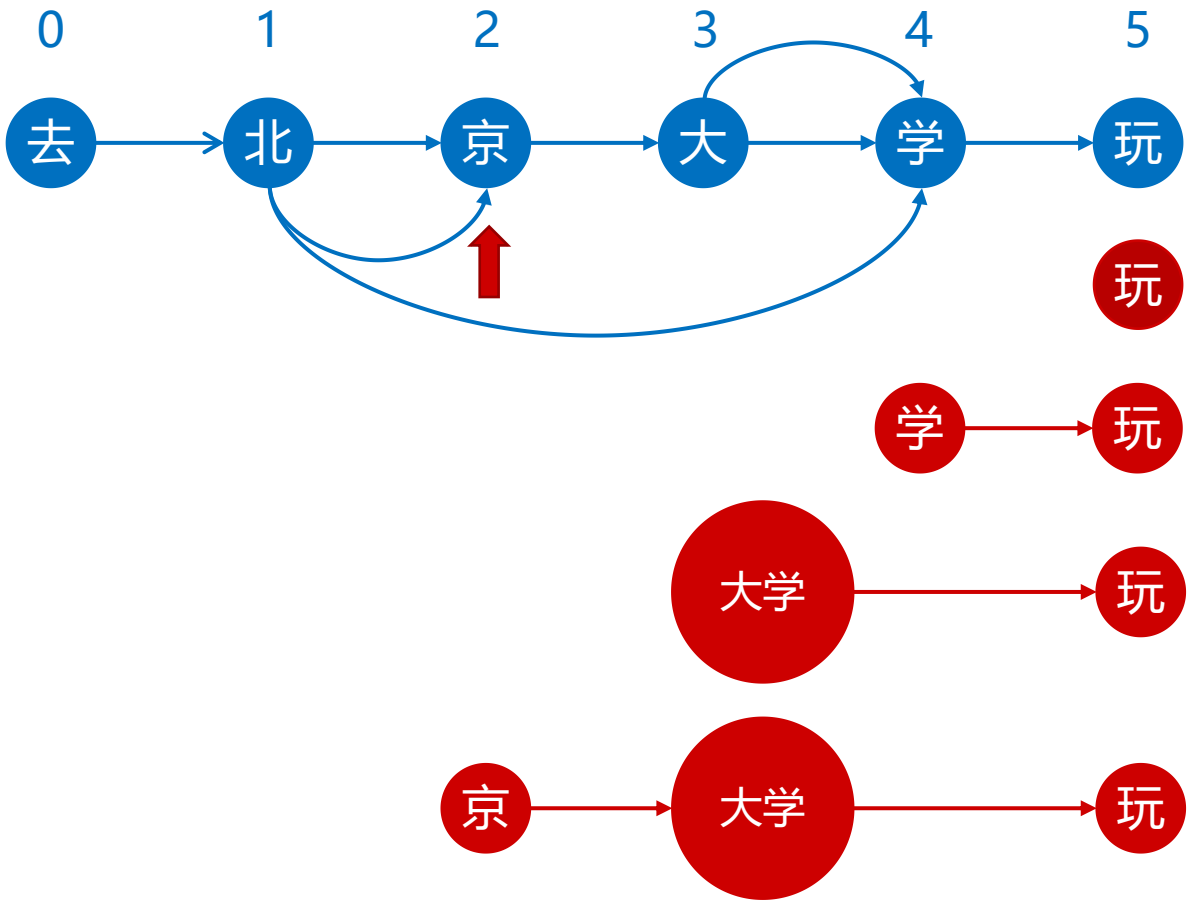


$p(\text{京}) =$ -9.12

$p(\text{京} \rightarrow \text{大学} \rightarrow \text{玩}) = -26.7$

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

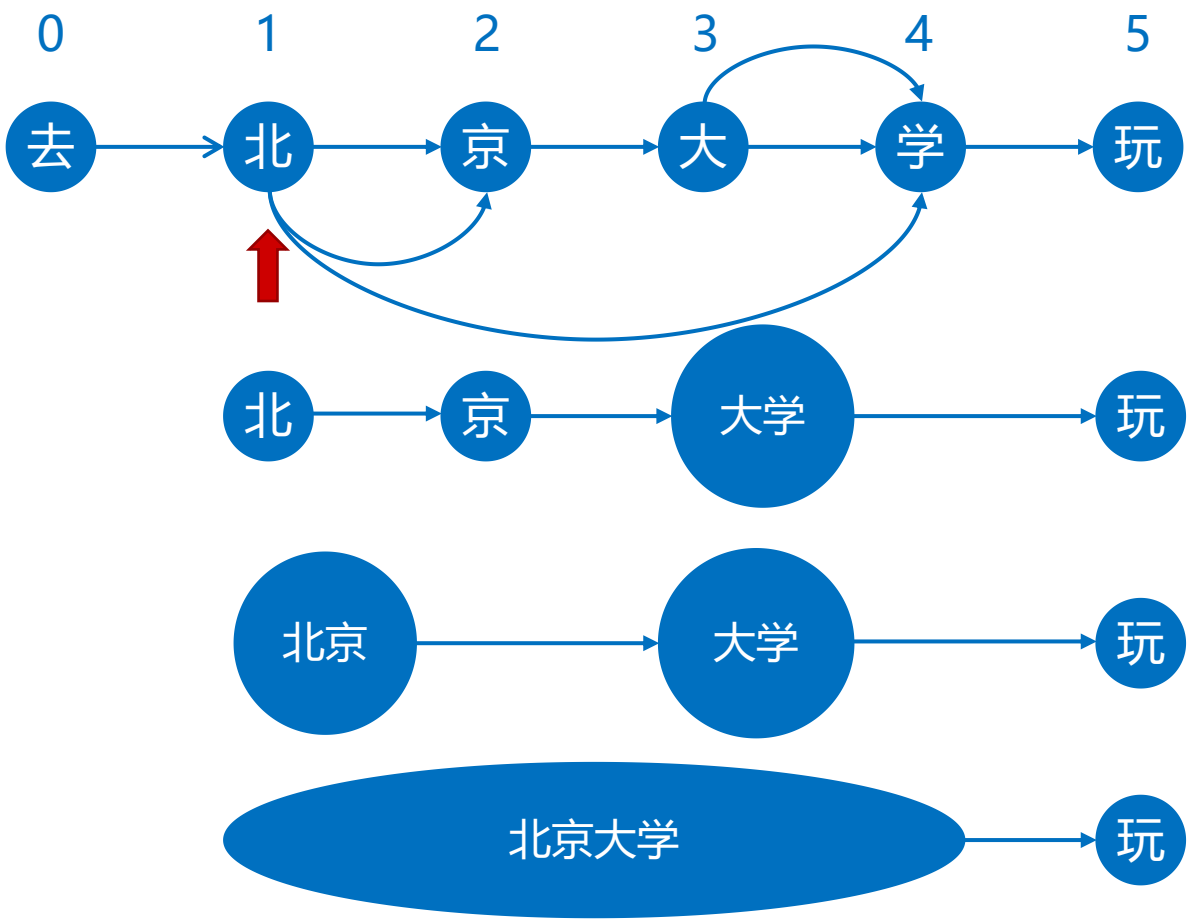
最大概率路径计算



Rounte ={6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4),
2: (-26.7, 2),
}

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

最大概率路径计算



Rounte ={6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4),
2: (-26.7, 2),
1: (-19.85, 4)
}

p(北) = -8.12

p(北 → 京 → 大学 → 玩) =− 34.82

p(北京) = -7.46

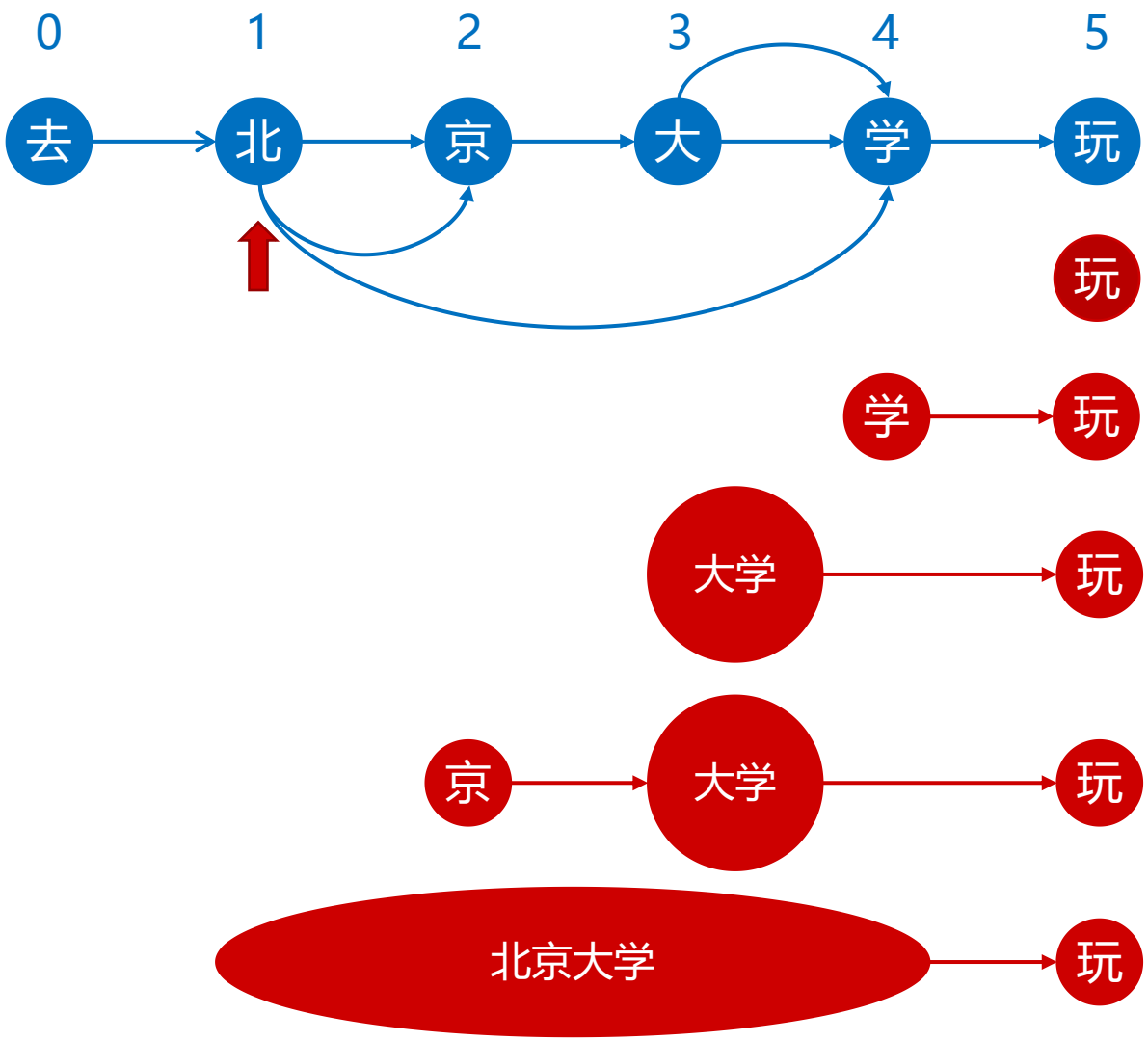
p(北京 → 大学 → 玩) =− 25.04

p(北京大学) = -10.28

p(北京大学 → 玩) = -19.85

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

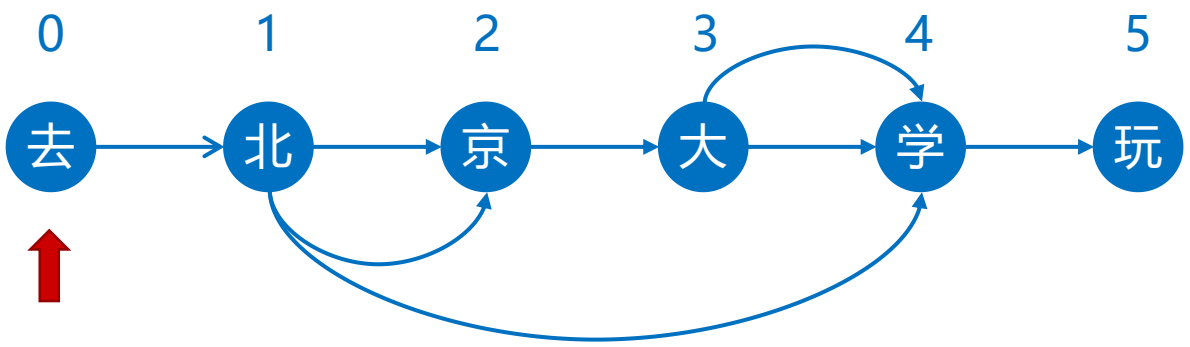
最大概率路径计算



Rounte ={6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4),
2: (-26.7, 2),
1: (-19.85, 4)
}

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

最大概率路径计算



Route = {6: (0, 0),
5: (-9.57, 5),
4: (-17.71, 4),
3: (-17.58, 4),
2: (-26.7, 2),
1: (-19.85, 4),
0: (-26.04, 0)
}

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

$p(去) = -6.19$

$p(去 \rightarrow 北京大学 \rightarrow 玩) = -26.04$

最大概率路径计算

```
def calc(sentence, DAG, route):
    N = len(sentence)
    route[N] = (0, 0)
    logtotal = log(total)
    for idx in range(N-1, -1, -1):
        tmp = []
        for x in DAG[idx]:
            # 计算概率值
            prob = log(FREQ.get(sentence[idx:x + 1]) or 1) - logtotal
            value = prob + route[x + 1][0]
            tmp.append( (value, x) )
        route[idx] = max(tmp)
```

0: [0]
1: [1,2,4]
2: [2]
3: [3,4]
4: [4]
5: [5]

{6: (0, 0), 5: (-9.57, 5), 4: (-17.71, 4), 3: (-17.58, 4), 2: (-26.7, 2), 1: (-19.85, 4), 0: (-26.04, 0)}

获取分词结果

```
def __cut_DAG_NO_HMM(sentence, f):
    DAG = get_DAG(sentence, f)
    route = {}
    calc(sentence, DAG, route)
    x = 0
    N = len(sentence)
    buf = []
    result = []  # 用于存储最终的分词结果
    while x < N:
        y = route[x][1] + 1
        l_word = sentence[x:y]
        # 如果是连续的英文字母或数字进行合并
        if re_eng.match(l_word) and len(l_word) == 1:
            buf.append(l_word)
            x = y
        else:
            if buf:
                result.append(''.join(buf))
                buf = []
            buf.append(l_word)
            x = y
    # 如果句子以连续英文或数字结尾
    if buf:
        result.append(''.join(buf))
    return result
```